

SELF-PROMPT LINEAGE

P0

- **State that generated it:** After implementing the new joint-observation logic and typed `SelectionReason`, the system still returned a free-form “Reason” string and did not yet bind the witness to a specific candidate set or gate. We had a partially working `SelectConsequenceRelevantObservations` but its conclusions (`DO_NOT_OBSERVE`) were not fully justified by explicit data.
- **Why it was interesting:** We needed to ensure the `DO_NOT_OBSERVE` outcome truly meant “no tested observation in *this specific set* can alter dispatch,” not a vague or global claim. That required a structured witness containing hashes or IDs of the inputs. Without it, “status donation” (treating unknowns as if they proved noninterference) could slip through.
- **What it caused:** We reworked the selector to emit a typed `ObservationSelectionWitness` object instead of a reason string. The code now captures the identity of the basis (previous state), policy, gate implementation, and the exact candidate observations evaluated. For each candidate, it records admissible outcomes, any excluded/unknown cases, and the resulting gated dispatch for each outcome.
- **What reality taught:** The change revealed that the absence of this structure was allowing subtle mistakes. Once `DO_NOT_OBSERVE` is always accompanied by a complete witness (including hashed ID of the candidate set and gate), we can distinguish between “no candidates provided,” “all tested candidates noninterfering,” and “some candidates unresolved.” We saw that the old path where unknown outcomes could slip by was indeed dangerous: after the fix, cases that were previously (incorrectly) treated as safe now correctly block the `DO_NOT_OBSERVE`.
- **Why it expired:** Once the typed witness and candidate-set binding were in place and verified, this prompt’s goal was achieved. We now have a system that never concludes **no observation needed** without enumerating its proof. The issue of unbound rationale is closed, so P0 no longer applies.

P1

- **State that generated it:** With a robust witness framework in place, we noticed the selector’s status enum still lacked an explicit “unresolved” outcome. The `UNKNOWN` results from individual `EvaluateObservationRelevance` calls were being lumped together or ignored.
- **Why it was interesting:** We realized that “not proven relevant” is not the same as “proven irrelevant.” If an observation’s effect on dispatch is undetermined (due to excluded cases or partial info), the selector shouldn’t treat it as noninterfering. Conflating the two was the exact status-donation defect we warned about.
- **What it caused:** We added a new enum case `CANDIDATE_RELEVANCE_UNRESOLVED` (or similar) in `MinimumObservationResult`. Now, when any tested outcome is unknown or excluded, the selector yields a distinct status. We also adjusted the logic so that the presence of even one unresolved candidate blocks the `DO_NOT_OBSERVE` conclusion entirely.
- **What reality taught:** This change highlighted the difference between “proven safe” and “not yet proven.” In practice, we found test cases where an observation partly split admitted worlds but had

unknowns; under the new code, those cases no longer incorrectly silence further questioning. The selector now explicitly says “I can’t decide – need to observe or resolve more.”

- **Why it expired:** After implementing and testing this, the prompt’s requirement is met: unknown outcomes no longer silently donate irrelevance. The system now distinguishes all three situations (no candidates, none discriminating, some unresolved), so P1’s purpose is fulfilled.

P2

- **State that generated it:** The next missing piece was evidence that we were identifying the correct minimal observation. So far we could declare `DO_NOT_OBSERVE` with a witness, but we had not yet built the retention/discard-counterexample machinery. In other words, we needed to prove **minimality**: every discarded observation truly can be omitted, and every retained one truly is needed.
- **Why it was interesting:** Achieving “minimal necessary information” was central to our goal. This goes beyond filtering single observations: it requires showing for each candidate either a counterexample (two admitted worlds that differ on it and lead to different dispatch) or a proof of invariance (all outcomes give the same result). Without these, our notion of minimality is just a guess.
- **What it caused:** We began designing two parallel assays. For **retention witnesses**, we would generate pairs of admitted realities that differ only in a given observation’s value and verify they yield different `GateConsequenceQuotient` decisions. For **discard witnesses**, we would check every admissible outcome of an observation against the base dispatch to confirm invariance. We sketched data structures to record these counterfactual tests in the witness output.
- **What reality taught:** Conceptually, this step crystallized the dual role of the selector: not only must it *find* distinguishing observations, it must also *justify* why others can be ignored. In exploring test cases, we saw some observations that seemed redundant only because our admission predicate was too strict; fixing that changed which observations needed witnesses.
- **Why it expired:** This prompt is still in progress—it defines the next major implementation task. Once we have code that produces these retention/discard proofs for every evaluated observation, P2 will have been realized. So it doesn’t “expire” yet, but it sets up the subsequent development focus.

P3

- **State that generated it:** We already have single-observation selection, but we must handle cases where **combinations** of observations are jointly needed. The code currently evaluates candidates independently, but there might be cases where one observation alone is noninterfering but together with another becomes discriminating.
- **Why it was interesting:** Joint observation selection is known to break greedy algorithms. An example is if observing either `01` or `02` alone doesn’t change dispatch, but observing both does. If our system isn’t checking that, it could miss necessary info. This was hinted at by related work in sensor-selection and feature acquisition.
- **What it caused:** We constructed an XOR-style test: admitted worlds such that `01` splits them in one way and `02` in another, but only the combined information distinguishes the lawful successor. The existing code failed this test (classifying both as `NONINTERFERING` individually). We then updated the algorithm to consider small subsets of candidate observations (starting with pairs) when deciding.
- **What reality taught:** The exercise confirmed that “cheapest single-observation selection” is not sufficient. The system needed a loop: if all singletons were noninterfering, it would try pairs, etc. The

first attempt (naively checking pairs) was very expensive, so we also learned the need for pruning: if two observations are completely independent (in the sense of their partitions on admitted worlds), we can skip some combinations.

- **Why it expired:** Once joint-effect cases were handled correctly and tested (so that `01` or `02` alone yields NO, but `[01, 02]` yields YES), this prompt's immediate goal was met. The focus can now shift to optimization (finding minimal sets) rather than discovering new failure modes in pure logic.

PROMPTS THAT FAILED PRODUCTIVELY

- **Greedy Observation Selection (Initial Approach):** At first, we tried simply picking the lowest-cost discriminating observation and ignoring joint sets. This failed to catch multi-feature requirements, but it was productive: it highlighted the need for pairwise testing, and it resulted in discussions about XOR-tests and the limitations of greedy heuristics.
- **Silent Acceptance of Empty Candidates:** We tested the case of no observations provided. It turned out the code would incorrectly say `DO_NOT_OBSERVE` if the `candidate_set` list was empty. This "failure" taught us to explicitly handle empty input (leading to a `NO_CANDIDATES_SUPPLIED` status) and not to allow unwarranted conclusions from an empty set.
- **Assuming Observations Always Admissible:** We momentarily acted as if any suggested observation could be asked. Testing scenarios where an observation might violate authority or be out-of-scope caused a re-think: the system must admit or reject the **acquisition of the information itself**. Though we hadn't coded this yet, these considerations will shape the next iteration.

PROMPTS THAT GENERATED MOST INFORMATION

- **Joint Observations Synergy Prompt:** Constructing the four-world XOR example (as in P3) generated a lot of insight. It exposed the gap between individual relevance and combined relevance, pushing us to think in terms of information sets and minimal sufficient sets. This was more insightful than any single-candidate test.
- **Witness Structure Definition:** When we decided to formalize `ObservationSelectionWitness` (P0/P1), enumerating all its fields (basis, policy, candidate IDs, costs, outcomes, etc.) clarified precisely what data we needed to record. That exercise effectively documented the entire selection process in a way we hadn't clearly laid out before.

PROMPTS THAT GENERATED MOST ACTIVITY BUT LITTLE INFORMATION

- **Cost-based Selection Optimization:** The idea of choosing an observation by cost-to-value ratio (cost divided by expected dispatch refinement) led to considerable coding and discussion, but we realized it's essentially an implementation detail. The intellectual gain was modest compared to the effort; existing decision-theory literature already covers it. We can circle back after the core logic is solid.
- **Iterative vs Batch Observation Sets:** We briefly attempted to implement a full minimal-set solver (looking at all subsets of candidates) prematurely. This created a lot of code paths and potential optimizations (pruning, dominance), but without new higher-level insight yet. It showed that a full solver is expensive, and hinted that heuristics will be needed, but we deferred deep work here.

QUESTIONS THAT DID NOT EXIST AT THE START

- **“How do we prove the minimality of information retention?”** Initially, we were only gating single observations. The question of proving that *no smaller set of observations could suffice* was not on our radar until we began implementing the DO_NOT_OBSERVE logic.
- **“Should observation acquisition itself obey admission rules?”** The idea that *asking a question* might need authority or permission was a late realization. We initially treated all candidate observations as given, but then recognized we need to consider whether the system is allowed to perform them.
- **“What if two observations individually yield no change but together do?”** This combined-case consideration (the XOR scenario) only emerged after implementing and testing the independent cases. It became a new question for us to answer.

MODEL CHANGES

- The system’s **status enum** was extended (adding `CANDIDATE_RELEVANCE_UNRESOLVED`).
- The **selection result** now returns a structured witness object (`ObservationSelectionWitness`) instead of an untyped message.
- The logic of `SelectConsequenceRelevantObservations` was refactored to treat *unknown* outcomes differently from *noninterfering* ones, enforcing fail-closed behavior.
- A new capability for **pairwise (and in principle n-wise) observation testing** was sketched and partially implemented.
- Planning for **witness-based proofs** (retention vs discard) was introduced, although the code for generating those proofs is still forthcoming.

ROOT CAUSES DISCOVERED

- A key root cause of errors was **status donation**: assuming that “not proven relevant” implied “proven irrelevant.” We fixed this by refusing to let `UNKNOWN` be treated as `NONINTERFERING`.
- We also uncovered that earlier designs treated the witness (and by analogy, the state) too abstractly. Binding the witness to specific input sets and gate versions is necessary to prevent stale or out-of-context claims.
- Another realization was that the notion of “preserving all admitted distinctions” was too coarse. The real need is “preserving exactly those distinctions necessary for lawful dispatch.”

GENERALIZATIONS KILLED

- We had assumed that a single stage of observation checking would suffice. The XOR counterexample killed that generalization, proving we need multi-step (possibly backtracking) checking.
- Initially, we thought of observations as always being freely available information. We discarded that generalization once we considered governance and authoritativeness of data sources.
- We also let go of the idea that the admission and disposition mechanisms could be merged into one monolithic “state engine.” The realization that there are two runtime kernels (Admission vs Disposition) separated this concern, so we killed the earlier monolithic generalization.

GENERALIZATIONS THAT SURVIVED

- The pipeline architecture
(candidate → admission → transition → disposition → next state) remains valid and central.
- The approach of separating byte-level authenticity, authoritativeness, and semantic correctness as distinct constraints still holds; these layers continue to map onto our code modules (replay, admission, canon, etc.).
- The principle of **bounded reasoning**—only using explicitly provided information—remained intact. Every time we hesitated before inferring beyond the code, that discipline kept our design conservative.

CROSS-DOMAIN CONNECTIONS EARNED

- We drew on **sensor/feature selection theory** (identifying minimal features for a decision) to shape the joint-observation logic. This helped us see our problem as akin to *active feature acquisition*, a well-studied area.
- **Bisimulation and coarsest partition algorithms** informed the idea of `GateConsequenceQuotient`: we are effectively merging worlds that are equivalent with respect to the chosen consequence.
- The notion of **value-of-information** from decision theory surfaced when we discussed cost-based selection: an observation should only be asked if its benefit (reducing dispatch uncertainty) outweighs its cost.
- The **capsule's verification model** served as an example of fail-closed reasoning. Our use of explicit unimplemented checks and withheld proofs came directly from how the capsule refused to claim “canonical semantic equivalence” without explicit tests.

PATENT TOPOLOGY CHANGES

- The initial high-level patent outline favored large “State Admission Engine” claims. The ongoing work has shifted to **fine-grained computational claims** (e.g. “Consequence Admission Method,” “Disposition Closure Verification,” etc.). We’re essentially rearranging the patent families to match the emerging code primitives rather than abstract metaphors.
- Some claimed inventions from early discussions (like treating all admitted objects as “distinctions” to conserve) have faded in importance. The real innovations now seem to be about *how to compute admission/disposition*, not about *why* from a constitutional perspective.
- New inventions have appeared: *admission of consequence-bearing distinctions*, *joint-observation minimality*, and *proof-carrying abstention*. These were not in the original patent sketches but are now focal points.

WHAT YOUR PROMPT GENERATOR LEARNED

- The prompt generator (ourselves) has learned to focus sharply on **computational questions** that arise from the current system state, rather than abstract ideas. For example, asking “Why can’t we claim DO_NOT_OBSERVE without a witness?” led directly to the solution, whereas a philosophical question like “What is the nature of a legal state?” would not.

- We have learned to treat every *new code symptom* (failing test, logged warning, missing case) as a clue for the next prompt. For instance, the discovery of a failed joint-observation test immediately turned into a prompt about multi-observation relevance.
- The iterative prompting process taught us to prioritize minimal, well-scoped questions. Early on we might have tried to solve too much at once (all observations sets, all policies) and got nowhere. Narrowing prompts down to one specific mechanism at a time produces progress.

CURRENT SELF-PROMPT

“What is the minimal set of consequence-bearing observations needed to ensure correct dispatch for the current gate, and how can we prove that no smaller set would suffice under the admitted state?”

WHAT YOU ARE NOW TRYING TO MAKE REALITY ANSWER

We are now attempting to *compute and verify* the minimal observation set that determines lawful execution. In practical terms, the machine must identify exactly which questions (observations) it must ask so that the consequence admission (`GateConsequenceQuotient`) yields the same decision it would if it knew the full state. At the same time, we must generate proof evidence: showing that removing any one of these observations would change the dispatch outcome (necessity) and that adding any discarded observation would not (sufficiency). Reality needs to provide a construction (an algorithm and test cases) of this minimal-information frontier so that the runtime can “know no more—and no less—than what is necessary” to proceed.

Why this matters now: The code and tests currently verify individual and pairwise observation selection but have not yet established global minimality. The next step is to make the quotient semantics and relevance criterion two sides of the same theorem. Only after this completion will we be confident that the system’s information strategy is both sound and complete.
